

Step 1 — FastAPI Route — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

api/schemas.py

```
from pydantic import BaseModel

class TaskRequest(BaseModel):
    topic: str

class TaskResponse(BaseModel):
    final_report: str
    approved: bool
```

api/routes.py

```
from fastapi import APIRouter
from api.schemas import TaskRequest, TaskResponse
from graph import graph
from state import make_initial_state

router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest):
    result = graph.invoke(make_initial_state(request.topic))
    return TaskResponse(final_report=result["final_report"], approved=result["rev"])

@router.get("/health")
def health():
    return {"status": "ok"}
```

api/main.py

```
from fastapi import FastAPI
from api.routes import router

app = FastAPI(title="ContentForge API")
app.include_router(router)
```

Run with `uvicorn api.main:app --reload`, then hit `/docs` or `curl -X POST localhost:8000/run-task -H "Content-Type: application/json" -d '{"topic": "electric bikes"}'`. This works. A missing or crashed graph run currently returns FastAPI's default `500` with the full traceback in the response body — fine to notice now, not yet fixed.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, split into the standard FastAPI project shape

`api/schemas.py`

```
from pydantic import BaseModel, Field

class TaskRequest(BaseModel):
    topic: str = Field(..., min_length=1, description="The topic to research and")

class TaskResponse(BaseModel):
    final_report: str
    approved: bool
```

`api/routes.py`

```
from fastapi import APIRouter
from api.schemas import TaskRequest, TaskResponse
from graph import graph
from state import make_initial_state

router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest) -> TaskResponse:
    result = graph.invoke(make_initial_state(request.topic))
    return TaskResponse(
        final_report=result["final_report"],
        approved=result["review_verdict"].approved,
    )

@router.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

api/main.py

```
from fastapi import FastAPI
from api.routes import router

app = FastAPI(title="ContentForge API", version="0.1.0")
app.include_router(router)
```

Difference from Basic: `Field(..., min_length=1)` rejects a request with `topic: ""` at the validation layer — an empty-but-technically-present string that the Basic version's plain `str` type hint would let straight through, straight into Step 1 of Project 4's own "notes cannot be empty" guard clause deep inside the graph, where the error is far harder to trace back to a bad API request. Full type hints on every function. `app.include_router(router)` separates app setup (`main.py`) from route definitions (`routes.py`), which is what makes adding `GET /runs/{id}` in Step 2 a change to one file, not a restructure. Still not handled: what a client actually sees if `graph.invoke(...)` itself raises, and what happens to other requests while one is running.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — safe error handling, logged detail server-side

```
import logging
from fastapi import APIRouter, HTTPException
from api.schemas import TaskRequest, TaskResponse
from graph import graph
from state import make_initial_state

logger = logging.getLogger("contentforge.api")
router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest) -> TaskResponse:
    try:
        result = graph.invoke(make_initial_state(request.topic))
    except Exception:
        logger.exception("run-task failed for topic=%s", request.topic)
        raise HTTPException(status_code=500, detail="Task failed – please try again")

    return TaskResponse(
        final_report=result["final_report"],
        approved=result["review_verdict"].approved,
    )
```

Expected behavior on a real crash (e.g. bad `OPENAI_API_KEY`): the client receives `{"detail": "Task failed – please try again"}` with a `500` status — no file paths, no Python traceback, no internal variable names. The real exception and its full traceback are written to the server's own logs via `logger.exception(...)`, so you (not the client) can actually debug it.

Approach 2 — off the event loop, async route with `run_in_threadpool`

```
import logging
from fastapi import APIRouter, HTTPException
from starlette.concurrency import run_in_threadpool
from api.schemas import TaskRequest, TaskResponse
from graph import graph
```

```

from state import make_initial_state

logger = logging.getLogger("contentforge.api")
router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
async def run_task(request: TaskRequest) -> TaskResponse:
    initial_state = make_initial_state(request.topic)
    try:
        result = await run_in_threadpool(graph.invoke, initial_state)
    except Exception:
        logger.exception("run-task failed for topic=%s", request.topic)
        raise HTTPException(status_code=500, detail="Task failed – please try aga

    return TaskResponse(
        final_report=result["final_report"],
        approved=result["review_verdict"].approved,
    )

@router.get("/health")
async def health() -> dict[str, str]:
    return {"status": "ok"}

```

Expected behavior under concurrent load: a second `curl` request against `/health` (or another `/run-task`) fired while the first `/run-task` is still mid-flight now gets a response immediately — the blocking `graph.invoke(...)` call runs in a worker thread, not on the event loop that also has to answer every other request.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's route works, but a real failure leaks internals to the client, and a slow request blocks every other request behind it — invisible with one request in testing, real under actual traffic. Approach 1 fixes the first problem alone: it's a synchronous route, still blocking, but now fails safely. Approach 2 fixes both — the route becomes `async def` and off-loads the blocking call via `run_in_threadpool`, on top of the same safe error handling.

Which one should you actually write? Approach 2. The safe error handling in Approach 1 is non-negotiable regardless of concurrency — never skip it. But `run_in_threadpool` is what actually makes this a service rather than a script wearing an HTTP route: FastAPI's whole performance model assumes routes don't block the event loop, and `graph.invoke(...)` — several LLM calls, possibly multiple writer/reviewer rounds — is exactly the kind of slow, synchronous call that assumption breaks on if you don't handle it explicitly.